

packages/graph-view/src/utils/layerStats.ts

Kind: added · Baseline: a9c34bfb03fe · Target: NovaDiff · Generated: 2026-05-20T05:48:14.228Z

Overview

A new utility file `packages/graph-view/src/utils/layerStats.ts` (lines 1-39) adds logic to compute statistics for a graph layer. It introduces a `Complexity` type, a `LayerStats` interface, and the `computeLayerStats` function.

Key changes

- `import type { GraphNode, Layer } from "@novadiff/graph-core/types";` (R1) pulls core graph types.
- `export type Complexity = "simple" | "moderate" | "complex";` (R3) defines three complexity buckets.
- `export interface LayerStats { resolvedCount: number; aggregateComplexity: Complexity; }` (R5-10) exposes the result shape.
- `export function computeLayerStats(layer: Layer, nodesById: Map<string, GraphNode>): LayerStats` (R20-39) iterates over `layer.nodeIds`, counts resolved nodes, tallies complexity buckets, and derives an aggregate label when a bucket exceeds 30% of resolved nodes (threshold logic in R32-37).

The implementation runs in **$O(\text{layer.nodeIds.length})$** , replacing the former $O(N \times K)$ filter described in the comment block (R12-19).

Impact

- **Performance:** Eliminates the super-linear cost of filtering nodes per layer, reducing render times for large graphs.
- **API surface:** Provides a typed `LayerStats` object and a clear `Complexity` type, improving type safety.
- **Maintainability:** Centralizes complexity logic, reducing duplication.
- **Observability:** `resolvedCount` can be logged or displayed to detect incomplete node mappings.

Risks & follow-ups

- **Node mapping correctness:** If `nodesById` lacks entries, `resolvedCount` will be lower than expected; verify that all layers receive a complete map.
- **Threshold logic:** The 30% rule is hard-coded; confirm it matches business requirements and update tests if thresholds change.
- **Integration:** Ensure existing code that relied on the old $O(N \times K)$ logic now imports and uses `computeLayerStats` correctly.